TEMA 3. Excepciones

Empecemos un tema sobre control de errores en lenguajes de programación, con algo básico. En C, donde no existen las excepciones, pongamos un ejemplo de una raíz que toma número flotante positivo.

1. Queremos controlar el error si la función recibe un número negativo. El usuario debe ser informado pero desde fuera de la función `raiz` ¿Cómo indicamos ese error?. Enumera dos opciones diferentes de diseñar, poniendo un ejemplo de código de cada una.

Opción 1: Usar un código de retorno especial

En C, una estrategia común es devolver un valor especial que indique un error. Por ejemplo, se puede devolver -1 o NaN (Not a Number) para señalar un fallo. El código llamador debe verificar este valor especial después de cada llamada. Esta aproximación es simple pero poco elegante, ya que el valor especial más significativo (el resultado) se mezcla con la información del estado (éxito o error).

```c
#include <math.h>
#include <stdio.h>

float raiz(float numero) {
    if (numero < 0) return -1.0;  // Indica error
    return sqrt(numero);
}

int main() {
    float resultado = raiz(-4);
    if (resultado == -1.0) {
        printf("Error: no se puede calcular raíz de número negativo\n");
    }
    return 0;
}
```

Opción 2: Usar un parámetro de salida y devolver un código de estado

Otra alternativa es pasar un puntero donde la función almacena el resultado, y devolver un código de error (0 para éxito, -1 para fallo). De esta forma se separa claramente el valor calculado del estado

de la operación. Esta aproximación es más robusta porque el resultado y el estado viajan por caminos diferentes, pero requiere más verbosidad: se debe pasar una dirección de memoria y verificar después si la operación fue exitosa.

```c
#include <math.h>
#include <stdio.h>

int raiz(float numero, float *resultado) {
    if (numero < 0) return -1;  // Error
    *resultado = sqrt(numero);
    return 0;  // Éxito
}

int main() {
    float resultado;
    int estado = raiz(-4, &resultado);
    if (estado == -1) {
        printf("Error: no se puede calcular raíz de número negativo\n");
    }
    return 0;
}
```

2. Brevemente ¿Qué es una "excepción"? ¿Con qué objetivo las usa un programador cuando implementa funciones o cuando las llama?

Una excepción es un evento extraordinario que representa una situación anómala durante la ejecución de un programa, como intentar acceder a un archivo inexistente, dividir por cero o asignar memoria inválida.

Se utiliza `throw` para señalar que ha ocurrido una condición de error, permitiendo que el código llamador maneje la situación de forma explícita mediante `try-catch`.
El programador que llama la función puede entonces decidir cómo reaccionar ante la excepción: captarla y recuperarse, registrar el error, o permitir que se propague a niveles superiores.

3. Reescribe el mismo ejemplo de raiz, pero en Java, metiendo ese método en una clase `Calculadora` y

llama a dicho método desde el método `main`, mostrando cómo se puede controlar desde fuera.

```
public class Calculadora {
    public double raiz(double numero) {
        if (numero < 0) {
            throw new IllegalArgumentException(
                "No se puede calcular raíz de número negativo"
            );
        }
        return Math.sqrt(numero);
    }
}

public class Main {
    public static void main(String[] args) {
        Calculadora calc = new Calculadora();

        try {
            double resultado = calc.raiz(-4);
            System.out.println("Raíz: " + resultado);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

En Java, cuando el método `raiz()` detecta un número negativo, lanza una excepción usando `throw`. Esta excepción interrumpe la ejecución del método y salta al bloque `catch`.

El código que llama la función utiliza la estructura `try-catch`:

- El bloque `try` contiene el código que podría lanzar una excepción.
- El bloque `catch` especifica qué tipo de excepción se captura y cómo se maneja.
  En este caso, se imprime el mensaje de error encapsulado en la excepción, permitiendo que el programa continúe de forma controlada en lugar de terminar abruptamente.

4. Explica con el mismo ejemplo anterior en Java de la raíz cuadrada:

a) ¿Qué es "lanzar" una excepción?

Lanzar una excepción significa ejecutar la instrucción `throw`, creando un objeto excepción y deteniendo inmediatamente la ejecución del método.
En el ejemplo anterior, cuando se invoca `calc.raiz(-4)` y la función detecta un número negativo, se ejecuta `throw new IllegalArgumentException(...)`, lo que crea el objeto excepción y abandona el método sin devolver ningún valor.

b) ¿Qué es "controlar" o "capturar" una excepción?

Controlar o capturar una excepción es el acto de interceptarla usando un bloque `catch`. En el `main`, el bloque `catch (IllegalArgumentException e)` captura la excepción que fue lanzada en `raiz()`, permitiendo ejecutar código alternativo como imprimir un mensaje de error. Sin este bloque, el programa finalizaría abruptamente con un error no controlado.

c) ¿Qué es que se "propague" una excepción?

Propagar una excepción significa que, si no se captura en el método donde se lanzó, la excepción "viaja" hacia arriba por la pila de llamadas buscando un `catch` que pueda manejarla.

d) ¿Qué le va ocurriendo a las funciones en la pila de llamadas por donde se va propagando la excepción?

Desde el punto de vista de la pila: cuando `raiz()` lanza la excepción, ese método desaparece de la pila inmediatamente; el control retrocede un nivel buscando un `catch` en `main()`, donde lo encuentra.

e) ¿Las funciones que no la controlan se reanudan después de alguna forma?

Las funciones intermedias que no capturan la excepción simplemente desaparecen de la pila sin reanudarse; la ejecución no retorna a ellas ni se ejecuta el código que hubiera después de la llamada. Solo el código dentro del bloque `catch` correspondiente se ejecuta después de que la excepción es capturada.

5. ¿Qué ventajas tiene frente a C, la "propagación natural" de las excepciones a través de la pila (*stack*) de llamadas?

La ventaja fundamental es la **claridad** y **mantenibilidad** del código: la lógica normal se separa claramente de la lógica de manejo de errores, y no existe riesgo de que se ignore accidentalmente un error por falta de verificación.

6. En orientación a objetos, ¿las excepciones suelen ser objetos?

Sí, en Java las excepciones son objetos que heredan de la clase `Throwable`. Cada excepción encapsula información relevante al error en sus atributos y métodos.

¿Qué ventajas tiene esto en términos de encapsulación?

La excepción encapsula en un objeto toda la información contextual del error.

¿Podemos entonces crear excepciones personalizadas?

Si, y estas pueden encapsular campos adicionales, constructores personalizados, etc.
Esto hace el código más semántico, fácil de mantener, y permite a quien llama la función ser muy específico sobre qué tipos de errores puede esperar.

7. ¿Qué **información esencial** lleva cualquier **objeto excepción** que es muy útil tener cuando se llega a un manejador?

- El **mensaje de error** (accesible mediante `getMessage()`), que describe qué salió mal de forma legible y contextualizada.
- La **traza de la pila** (accesible mediante `printStackTrace()` o `getStackTrace()`), que registra toda la cadena de métodos que se ejecutaron hasta el punto donde se lanzó la excepción (útil para depuración).
- Opcionalmente, una **causa** (referencia a otra excepción que originó ésta, accesible mediante `getCause()`).

Cuando se captura la excepción en el manejador, toda esta información está disponible encapsulada en el objeto.

8. En Java, sobre el bloque "try-catch", ¿se pueden tener más de un bloque `catch` ? ¿cuántos bloques `catch` se ejecutan?

Sí, se pueden tener múltiples bloques `catch` después de un único bloque `try` . Esto permite diferenciar el manejo según el tipo específico de excepción que se lance.
Cada bloque `catch` captura excepciones de un tipo determinado, y Java los evalúa en orden de arriba a abajo, ejecutando **solo el primer bloque `catch`** cuyo tipo sea compatible (por igualdad o por herencia) con la excepción lanzada.
Por eso se recomienda colocar los tipos más específicos primero (como `IOException`) y los más generales al final (como `Exception`).

```
try {
    // código que puede lanzar excepciones
} catch (AccessDeniedException e) {
    System.out.println("Acceso denegado: " + e.getMessage());
} catch (IOException e) {
    System.out.println("Error de entrada/salida: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Error general: " + e.getMessage());
}
```

9. Si las excepciones producen rupturas en el código llamador, ¿cómo podemos garantizar que se ejecuta siempre finalmente un código necesario para cierre de ficheros, liberacion de recursos, antes de que continúe propagándose la excepción? Pon un ejemplo en Java con `finally` , tanto con `catch` como sin él.

El bloque `finally` se ejecuta **siempre**, independientemente de si se lanzó una excepción o no, o con un return o sin él.
Es el mecanismo perfecto para colocar código de limpieza (cierre de ficheros, liberación de recursos, etc.) que debe ejecutarse en cualquier circunstancia. Incluso si se propaga una excepción, el bloque `finally` se ejecuta antes de que la excepción continúe subiendo por la pila de llamadas.

Ejemplo con `catch` :

```java
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("archivo.txt"));
    String linea = reader.readLine();
    System.out.println(linea);
} catch (IOException e) {
    System.out.println("Error de lectura: " + e.getMessage());
} finally {
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            System.out.println("Error cerrando fichero: " + e.getMessage());
        }
    }
}
```

Ejemplo sin `catch` (solo `try-finally`):

```java
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("archivo.txt"));
    String linea = reader.readLine();
    System.out.println(linea);
    // Si ocurre una excepción, se propaga después de finally
} finally {
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            System.out.println("Error cerrando fichero: " + e.getMessage());
        }
    }
}
```

En ambos casos, el bloque `finally` garantiza que el recurso se cierre correctamente. Sin `catch`, la excepción se propaga después de ejecutar `finally`, lo que permite usar `finally` como garantía de limpieza sin necesariamente manejar el error en ese nivel.

10. En Java, el bloque `finally` puede ir sin `catch`? ¿Se ejecuta siempre tanto si ocurre como si no

ocurre una excepción? ¿Y si hay un `return` en medio del `try` ?

Sí, el bloque `finally` puede ir sin `catch` .
El bloque `finally` se ejecuta **siempre**, sin excepciones (valga el juego de palabras). Tanto si el código del `try` se ejecuta correctamente como si lanza una excepción o si hay un `return` , `break` o `continue` en medio del bloque, el `finally` se ejecutará. Esto hace que sea el lugar perfecto para código de limpieza. Incluso si hay un `catch` que captura la excepción y el programa parece recuperarse normalmente, el `finally` igualmente se ejecuta después del `catch` .

```
public static int ejemplo() {
    try {
        return 1;  // Se marcará para retornar 1
    } finally {
        System.out.println("Se ejecuta finally incluso con return");  // Esto se imprime
        // Aquí podría haber return 2; que sobrescribiría el return 1
    }
}
```

11. En Java, qué son las excepciones "controladas" y las "no controladas"?

- Las excepciones **no controladas** (unchecked exceptions) heredan de `RuntimeException` y no requieren captura obligatoria. Representan típicamente errores de programación o situaciones que una vez solventados, no vuelven a ocurrir.
- Las excepciones **controladas** (checked exceptions) son aquellas que el compilador de Java **obliga** a tratar explícitamente mediante `try-catch` o `throws` . Típicamente errores por causas externas y que siempre pueden llegar a ocurrir, como errores de E/S.

¿Qué papel juega `RuntimeException` ?

El papel de `RuntimeException` es ser la clase base de todas las excepciones no controladas. Cualquier excepción que herede de `RuntimeException` se considera "no controlada", mientras que las que heredan directamente de `Exception` se consideran "controladas".

Pon un ejemplo de excepciones típicas controladas y no controladas que incluso nosotros mismos podríamos usar.

Ejemplos de excepciones controladas propias:

1. `FileNotFoundException` (fichero no existe)
2. `IOException` (error de lectura/escritura).

Ejemplos de excepciones no controladas propias:

3. `IllegalArgumentException` (parámetro inválido),
4. `NullPointerException` (acceso a referencia nula).

Se puede crear una excepción controlada heredando de `Exception` y una no controlada heredando de `RuntimeException`:

```
// Excepción controlada personalizada
public class UsuarioNoEncontradoException extends Exception {
    public UsuarioNoEncontradoException(String mensaje) {super(mensaje);}
}
// Excepción no controlada personalizada
public class ValorNegativoException extends RuntimeException {
    public ValorNegativoException(String mensaje) {super(mensaje);}
}
```

Haz dos listas con 3 o 4 ejemplos de situación donde se suele preferir una excepción controlada y donde se suele preferir una excepción no controlada.

Situaciones donde se prefiere excepción controlada:

- Acceso a ficheros en disco (el fichero podría no existir)
- Conexiones de red (la red podría estar caída)
- Lectura de datos de servidor remoto (el servidor podría ser inaccesible)
- Parseo de documentos XML o JSON externos (el formato podría ser inválido)

Situaciones donde se prefiere excepción no controlada:

- Validación de parámetros (parámetro negativo cuando se espera positivo)
- Operaciones con colecciones (índice fuera de rango)
- Precondiciones de un método (estado incorrecto del objeto)
- Errores derivados de bugs en el código (no se puede recuperar)

12. ¿Qué es y para qué se usa `throws` ? ¿Por qué es alternativa a capturar una excepción controlada?

`throws` es una palabra clave que se añade a la firma de un método para declarar que ese método puede lanzar una o más excepciones controladas,, transfiriendo la responsabilidad del manejo a un nivel superior de la aplicación.

```
public void abrirArchivo(String nombreArchivo) throws IOException {
    FileInputStream fis = new FileInputStream(nombreArchivo);
    // Si el archivo no existe, se lanza IOException sin capturarla
}
```

`throws` es **alternativa a capturar** porque Java obliga a tratar las excepciones controladas de una u otra forma: bien capturándolas con `try-catch` , o bien declarándolas en `throws` . No se puede simplemente ignorar una excepción controlada; el compilador rechaza el código que no la trate de alguna manera.

13. Pon un ejemplo en Java de firma de método que incluya `throws` , de una función que abre un fichero pero que declara que no le interesa manejar la excepción de si el fichero no existe, sino que se propague hacia arriba. Eso sí, acuérdate del `finally` .

```
public class LectorArchivos {
    public String leerPrimeraLinea(String nombreArchivo) throws IOException {
        BufferedReader reader = null;
        try {
            reader = new BufferedReader(new FileReader(nombreArchivo));
            return reader.readLine();
        } finally {
            if (reader != null) {
                try {
                    reader.close();
                } catch (IOException e) {
                    System.out.println("Error cerrando fichero: " + e.getMessage());
                }
            }
        }
    }
}
```

14. ¿Podemos poner en `throws` excepciones no controladas, como `RuntimeException` ? ¿Debería el método llamador entonces poner `try-catch` en ese caso? ¿Qué sentido tendría?

Por poder se puede, en ese caso el compilador no obligar a usar `try-catch` en el código llamador.

Tendría sentido exclusivamente por propósitos de documentación, aunque no es para nada lo habitual.

15. ¿Cuándo se recomienda usar excepciones controladas, como `IOException` , y cuándo no controladas como `IllegalArgumentException` ?

Se recomienda usar **excepciones controladas** cuando la situación de error es algo externo al programa que **potencialmente se puede recuperar**.

- Típicamente, un fichero no existe en el disco (pero podría crearse después), una conexión de red falla (pero podría restablecerse), etc.
- El programador que llama la función debe ser **consciente** de que existe una posibilidad de fallo y debe **decidir cómo reaccionar**: reintentar la operación, usar un valor por defecto, logging, etc.

Se recomienda usar **excepciones no controladas** cuando la situación representa un **error de programación** que indica que algo salió mal de forma inesperada o que no debería haber sucedido.

- Ejemplos: parámetro negativo cuando la función espera positivo (error en la validación del llamador), acceso a índice inválido en array (error lógico del programador), referencia nula cuando se esperaba un objeto válido.
- Estos errores generalmente no tienen recuperación posible en el punto donde ocurren; lo correcto es que el programador corrija su código. Por eso no tiene sentido obligar a capturarlas: hacerlo sería enmascarar bugs.

¿Existen en todos los lenguajes ambas opciones? En los que sólo existe una opción, ¿cuál es la más habitual?

En otros lenguajes de programación, la situación varía significativamente. Java es **único** al tener esta distinción explícita entre excepciones controladas y no controladas.
En lenguajes que **no tienen ambas opciones**, la más habitual es la **excepción no diferenciada/opcional**, similar a la de Python o C++. El razonamiento es que forzar el manejo de

ciertos tipos de excepciones comparte los riesgos del modelo de C (mucho código verificando errores) con los beneficios de usar excepciones.

16. Pon ejemplos de ambos casos.

¿Tiene sentido lanzar excepciones dentro del `catch` ?

Sí, tiene sentido en ciertos casos.

```
public class ProcesadorDatos {
    public void procesarArchivo(String ruta) throws DatosInvalidosException {
        try {
            BufferedReader reader = new BufferedReader(new FileReader(ruta));
            String linea = reader.readLine();
            int numero = Integer.parseInt(linea);
            reader.close();
        } catch (IOException e) {
            // Se transforma IOException en una excepción de negocio
            throw new DatosInvalidosException("No se pudo leer el archivo: " + ruta, e);
        }
    }
}

public class DatosInvalidosException extends Exception {
    public DatosInvalidosException(String mensaje, Throwable causa) {
        super(mensaje, causa);
    }
}
```

¿Se puede relanzar la misma excepción capturada? ¿Cuándo tendría sentido hacer esto?

Sí, se puede relanzar el mismo objeto excepción, tras, por ejemplo, haber ejecutado código en el catch.

```java
public class GestorTransacciones {
    public void procesarTransaccion() throws SQLException {
        Connection conexion = null;
        try {
            conexion = DriverManager.getConnection("jdbc:mysql://localhost/bd");
            // operaciones de base de datos
        } catch (SQLException e) {
            System.out.println("Error registrado: " + e.getMessage());
            if (conexion != null) {
                try { conexion.rollback(); }
                catch (SQLException ex) { /* ignorar */}
            }
            throw e;  // Se relanza la misma excepción original
        }
    }
}
```

En ambos casos, la idea es que el `catch` realice tareas necesarias (limpieza, logging, transformación) pero luego permita que la excepción continúe su camino hacia niveles superiores, garantizando que el error no se pierda o se oculte silenciosamente.

17. ¿En qué consiste que una excepción sea la "causa" de otra excepción? Pon un ejemplo en Java, donde capturemos una excepción de bajo nivel y la encapsulemos en otra personalizada de alto nivel. Cuando una excepción sale por pantalla y tiene una causa, ¿se ve?

Una excepción es la **"causa"** de otra cuando la segunda fue lanzada como consecuencia directa de la primera. En Java, esto se implementa a través de la **exception chaining** (encadenamiento de excepciones): se captura una excepción de bajo nivel y se lanza una excepción de más alto nivel, pasando la original como parámetro al constructor.
Esto preserva la información del error original manteniendo una referencia a través de `getCause()`, permitiendo rastrear toda la cadena de errores desde la causa raíz.

```java
// Excepción personalizada de alto nivel
public class ErrorOperacionBancaria extends Exception {
    public ErrorOperacionBancaria(String mensaje, Throwable causa) {
        super(mensaje, causa);
    }
}

public class TransferenciaManager {
    public void transferir(String origen, String destino, double cantidad) throws ErrorOperaci
        try {
            // código de conexión a base de datos...
        } catch (SQLException e) {
            throw new ErrorOperacionBancaria(
                "No se pudo completar la transferencia de " + cantidad + " desde " + origen,
                e // Se encadena la excepción de bajo nivel como causa
            );
        }
    }
}
```

Cuando una excepción sale por pantalla y tiene una causa, ¡sí se ve! Al ejecutar `e.printStackTrace()`, Java muestra tanto el error principal como la causa raíz. La salida tendría este aspecto:

```
ErrorOperacionBancaria: No se pudo completar la transferencia de 100.0 desde 1234
    at TransferenciaManager.transferir(TransferenciaManager.java:15)
    at Main.main(Main.java:22)
Caused by: com.mysql.jdbc.exceptions.jdbc4.SQLServerException: Connection timeout
    at com.mysql.jdbc.Connection.createNewIO(Connection.java:2435)
    at com.mysql.jdbc.Connection.<init>(Connection.java:1559)
    ... (más líneas de traza)
```

La sección `Caused by:` muestra explícitamente la excepción original que provocó el error. Esto es extremadamente útil para debugging.